

Atelier Préparation de Données Textuelles

Contexte

Une entreprise souhaite développer un système capable de classer automatiquement des avis clients afin d'identifier leur sentiment.

Les avis proviennent de plusieurs sources : site web, application mobile, formulaire de satisfaction, réseaux sociaux et service client.

Les données collectées sont cependant de qualité variable : textes vides, doublons, fautes de frappe, majuscules/minuscules, caractères spéciaux, emojis, URLs, mentions, répétitions de caractères, textes très courts ou très longs, plusieurs catégories de sentiment et quelques valeurs manquantes.

Avant de construire un modèle de Machine Learning ou de Deep Learning, les apprenants doivent donc construire un pipeline complet de préparation des données textuelles.

Objectifs pédagogiques

À la fin de l'atelier, l'apprenant doit être capable de :

- 1) explorer un corpus textuel ;
- 2) identifier les problèmes de qualité des données ;
- 3) nettoyer les textes ;
- 4) tokeniser les textes ;
- 5) normaliser les données textuelles ;
- 6) transformer les textes en représentations numériques ;
- 7) comparer plusieurs méthodes de vectorisation ;
- 8) préparer un corpus exploitable par un modèle de ML/DL.

Structure du projet

- 1) Structurer le projet comme suit :

```
atelier_prepa_donnees_textuelles/  
├── notebooks/  
│   └── atelier_prepa_donnees_textuelles.ipynb  
└── data/  
    ├── smart_reviews_raw.csv  
    ├── smart_reviews_cleaned.csv  
    └── tfidf_matrix.npz
```

- 2) Stocker le fichier fourni `smart_reviews_raw.csv` dans le sous-dossier `data`
- 3) Créer le notebook `atelier_prepa_donnees_textuelles.ipynb`
 - Le fichier `smart_reviews_cleaned.csv` sera créé juste après la normalisation et contiendra par exemple, `id_avis`, `date`, `source`, `produit`, `texte`, `texte_clean`, `texte_final`, `sentiment`, `note` et `langue`.
 - Le fichier `tfidf_matrix.npz` contiendra la sauvegarde de la matrice retenue juste après la vectorisation

Partie 1 – Exploration du corpus

- 1) Charger les données CSV ;
- 2) Combien d'avis contient le dataset ?
- 3) Combien de colonnes possède-t-il ?
- 4) Quel est le type de chaque colonne ?
- 5) Existe-t-il des valeurs manquantes ?
- 6) Identifier quelques types de texte en affichant par exemple : texte normal ; texte vide ; texte contenant une URL ; texte contenant une mention ; texte contenant un hashtag ; texte contenant des emojis ; texte avec beaucoup de ponctuation ; texte en majuscules ; texte avec répétition de caractères.
- 7) Mesurer la longueur des textes en créant une nouvelle colonne « `longueur` » pour déterminer : longueur minimale ; longueur maximale ; longueur moyenne ; médiane et quartiles.
- 8) Visualiser la distribution de la longueur des avis puis répondre aux questions
 - a) Existe-t-il des textes anormalement longs ?
 - b) Existe-t-il beaucoup de textes très courts ?
- 9) Détecter les textes vides
- 10) Détecter les doublons sur le texte
- 11) Les classes sont-elles équilibrées ?
- 12) Quelle est la classe majoritaire ?
- 13) Quelles métriques de classification à envisager d'utiliser ?
- 14) Détecter les caractères particuliers : emojis ; URLs ; hashtags ; mentions ; chiffres ; caractères spéciaux ; ponctuation répétée ; ...
- 15) Visualiser et commenter nombre d'avis par source
- 16) Visualiser et commenter le nombre d'avis par produit

Partie 2 – Nettoyage

- 1) Supprimer les lignes contenant des valeurs manquantes de texte
- 2) Supprimer les doublons
- 3) Supprimer les URLs
- 4) Traiter les mentions et hashtags avec la stratégie la plus pertinente pour une classification de sentiment
- 5) Nettoyer les espaces
- 6) Traiter la ponctuation avec la stratégie la plus pertinente pour une classification de sentiment
- 7) Créer une fonction de nettoyage pour automatiser les actions ci-dessus
- 8) Stocker sur une colonne `texte_clean`, les textes ayant subi les traitements ci-dessus
- 9) Vérifier le contenu des colonnes `texte` et `texte_clean` sur les 10 premières observations

Partie 3 – Tokenisation

- 1) Pourquoi `split()` n'est-il pas suffisant pour une vraie application NLP ?
- 2) Utiliser la bibliothèque `NLTK` pour déterminer :
 - a) le nombre de tokens de chaque texte,
 - b) nombre moyen de tokens ;
 - c) texte avec le plus grand nombre de tokens ;
 - d) texte avec le plus petit nombre de tokens.

Partie 4 – Normalisation

- 1) Uniformiser la casse en minuscules. Pourquoi ce traitement ?
- 2) Faut-il systématiquement supprimer les accents dans un corpus français ?
- 3) Quels stop words supprimer ?
- 4) Quels stop words conserver ?
- 5) Appliquer la technique (stemming ou lemmatisation) la plus adaptée ?
- 6) Stocker sur une colonne « `texte_final` », les textes ayant subi le nettoyage et les traitements ci-dessus
- 7) Vérifier le contenu des colonnes `texte`, `texte_clean` et `texte_final` sur les 10 premières observations
- 8) Sauvegarder le dataset nettoyé dans le fichier `smart_reviews_cleaned.csv`

Partie 5 – Découpage Train/Test

Diviser en deux ensembles distincts : un pour l'entraînement (`train`) et un pour le test (`test`). Avec les conditions suivantes : 20% des données serviront au test ; garantir la reproductibilité du découpage ; conserver les mêmes proportions de classes dans l'ensemble de train et de test que dans les données d'origine.

Partie 6 – Vectorisation

- 1) Utiliser la technique « **Bag of Words** » pour générer et afficher la taille et un extrait du vocabulaire ainsi qu'un morceau de la matrice
- 2) Utiliser la technique « **TF-IDF** » pour générer et afficher la taille et un extrait du vocabulaire ainsi qu'un morceau de la matrice
- 3) Utiliser la technique « **TF-IDF** avec des **unigrams et bigrams** » pour générer et afficher la taille et un extrait du vocabulaire ainsi qu'un morceau de la matrice
- 4) Comparer les trois techniques
- 5) Sauvegarder la matrice retenue dans le fichier `tfidf_matrix.npz`
- 6) Pourquoi éviter de faire certaines techniques de vectorisation avant le découpage du dataset ?

Partie 7 : Bonus

Proposer et réaliser toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'atelier.

Livrable attendu

Chaque apprenant devra fournir le dossier `atelier_prepa_donnees_textuelles/`

Modalités de livraison

Chaque apprenant pousse

- 1) le livrable, **d'abord vide**, sur un dépôt public de son compte Github personnel ;
- 2) met à jour le livrable **après chaque tâche accomplie** (sans attendre de terminer toute une partie), avec un message de commit explicite.